MODULE 35 · SECTION 5 - MAINTAINING AND AUTOMATING DATA WORKLOADS

Orchestration: Managed Airflow, Workflows and Scheduling

Choosing between Managed Airflow, Workflows and Cloud Scheduler; DAG design, retries and idempotency; sensors and deferrable operators; event-driven DAGs; generations; worker evictions.

SECTION 5 of 5 EXAM WEIGHT 18% MODULE 35 of 45

admin@sthithapragna.com

An orchestrator decides what runs, and when. The heavy lifting happens somewhere else.

admin@sthithapragna.com

Three ways to run things in order

TOOL	BEST FOR	WRITTEN IN
Managed Airflow	Data pipelines with dependencies, retries, backfills	Python DAGs
Workflows	Chaining APIs and services, low latency, serverless	YAML or JSON
Cloud Scheduler	One job on a timetable, such as a nightly trigger	A cron schedule

THE TIEBREAKER

Many dependent data steps, with reruns: Airflow. A few API calls, idle most of the time: Workflows.

admin@sthithapragna.com

Managed Airflow or Workflows?

MANAGED AIRFLOW

Data pipelines

- 150+ Google Cloud operators
- Backfills and re-runs built in
- Latency in seconds between tasks
- Provisioned: pays while it exists

WORKFLOWS

Service orchestration

- Connectors for Google APIs, any HTTP call
- Callbacks, and waits of up to a year
- Latency in milliseconds
- Serverless: nothing to pay when idle

REMEMBER

Google's own guide: Airflow for ETL and ELT, Workflows for chaining services.

Tasks that are safe to rerun

- Each task does **one step**, and is **idempotent**: running it twice gives the same result.
- Write by partition: **overwrite** the run's date, never append blindly.
- Set **retries** and a retry delay, so one blip does not fail the whole run.
- Use a **fixed start date**. A dynamic one changes every time the DAG is parsed.

> THE TRAP
>
> Retries, backfills and recovery all assume idempotent tasks. Without it, each rerun duplicates data.

Larkspur's nightly settlement DAG

```
1 with DAG('settlement_nightly', start_date=datetime(2026, 10, 1),
2          schedule='0 1 * * *', catchup=False,
3          default_args={'retries': 2, 'retry_delay': timedelta(minutes=5)}) as dag:
4     wait = GCSObjectExistenceSensor(task_id='wait_for_westgrid', deferrable=True,
5         bucket='lk-landing-prod', object='westgrid/{{ ds }}/settlement.csv')
6     settle = DataprocSubmitJobOperator(task_id='run_settlement', job=SETTLE_JOB)
7     marts = BigQueryInsertJobOperator(task_id='rebuild_marts', configuration=MARTS_SQL)
8     wait >> settle >> marts
```

2-3 Daily at 01:00, no automatic catch-up, two retries per task.

4 A deferrable sensor waits in the triggerer, not on a worker.

FOR LARKSPUR

The {{ ds }} date makes each run read and write only its own day, so reruns are safe.

A sensor, or a deferrable operator?

CLASSIC SENSOR

Holds a worker slot

- Polls on a schedule
- Occupies a worker the whole time
- Hundreds of waits fill the workers
- Fine for short waits

DEFERRABLE

Waits in the triggerer

- Hands the wait to the triggerer
- Frees the worker slot while waiting
- Survives environment updates
- Needs at least one triggerer

THE TIEBREAKER

Many long waits on files or jobs: deferrable operators. They are cheaper and more robust.

Catchup, backfill and failure alerts

- With **catchup** on, a new DAG runs every missed interval since its start date.
- Set catchup off, then **backfill** on purpose, for exactly the dates you need.
- An **on_failure_callback** can post to a channel or open an incident.
- Airflow 3, in Gen 3, adds backfills from the UI and **asset-based** scheduling.

> THE TRAP
>
> A start date a year ago, with catchup on, launches a year of runs on the first deploy.

Starting a DAG when a file lands

01

File lands

Westgrid writes to the landing bucket.

02

Eventarc

Fires on the object finalize event.

03

Cloud Run function

Calls the Airflow REST API.

04

DAG runs

schedule=None, with the event as its config.

HONEST LIMIT

With Private IP or VPC Service Controls, use the Pub/Sub approach instead of the function.

admin@sthithapragna.com

Keeping workers healthy

- The commonest cause of **worker pod eviction** is a task's memory spiking.
- Add worker memory, or lower **worker_concurrency** so each runs fewer tasks.
- Push heavy work out: **KubernetesPodOperator**, or BigQuery, Dataflow, Spark.
- Keep DAG files light. **Top-level code** runs on every parse, and slows everything.

> REMEMBER
>
> An evicted worker interrupts every task running on it. Idempotent, retriable tasks survive that.

Managed Airflow generations

GENERATION	INFRASTRUCTURE	AIRFLOW
Gen 3	Hidden: no cluster in your project	2 and 3, DAG processors scale
Gen 2	Autopilot GKE in your project	2 only
Legacy Gen 1	Standard GKE, scaled by hand	End of life, 15 September 2026

CURRENCY CHECK

Composer 1, 2 and 3 in older questions are these generations. The product is now Managed Airflow.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Many dependent steps, **backfill** needed	Managed Airflow
Chain a few **APIs**, idle most of the day	Workflows
Run one thing **every night**	Cloud Scheduler
Hundreds of sensors **fill the workers**	Deferrable operators
Start when a **file lands**	Eventarc, function, Airflow REST API
Reruns create **duplicates**	Idempotent tasks, overwrite by partition

THE PATTERN

Orchestrate, wait cheaply, and make every step safe to repeat.

admin@sthithapragna.com

> When a customer changes tariff, Larkspur must call the billing API, then the CRM API, then send a confirmation email.
>
> It happens a few hundred times a day, must start within seconds, and nothing else needs orchestrating.

What should the team use?

A. A new Managed Airflow environment, with one DAG run per change

B. Cloud Scheduler, calling each API every minute in turn

C. A Dataflow streaming job that makes the three calls in order

D. Workflows, started per change, with retries on each call

◆ PAUSE AND DECIDE

What should the team use?

A Managed Airflow

Seconds of latency, and paid for even when idle.

B Cloud Scheduler

A timetable, not a reaction to each change.

C Dataflow

A data pipeline engine for a handful of API calls.

D Workflows

Serverless, milliseconds to start, built-in retries.

THE SIGNAL

A few API calls, started per event, idle otherwise: Workflows.

> The settlement DAG appends each night's results to curated.settlement. After a retry last week, one day's results were loaded twice.
>
> Finance needs reruns of any day to be safe.

What should the team change?

A Turn off retries, so that a failed task is never run twice

B Add a deduplication query that runs once at the end of each month

C Make each run overwrite its own day's partition, not append

D Raise worker memory, so that the task never fails mid-load

◆ PAUSE AND DECIDE

What should the team change?

A No retries

Every blip now fails the run instead.

B Monthly dedupe

Wrong figures for up to a month.

C Overwrite by partition

Idempotent: one run or five, the same result.

D More memory

Failures have many causes. Reruns still duplicate.

THE SIGNAL

Reruns duplicate data: make the task idempotent, overwrite by partition.

> The landing DAGs use classic sensors to wait for 300 partner files, some for hours.
>
> Workers are full of waiting sensors, and real tasks sit queued behind them.

What should the team change?

A Raise the worker maximum, so that there is room for every sensor

B Switch the sensors to deferrable mode, with a triggerer running

C Lower the sensors' poke interval, so they finish checking sooner

D Move every partner file check into one large Python task

◆ PAUSE AND DECIDE

What should the team change?

A More workers

Pays for workers that only wait.

B Deferrable sensors

Waiting moves to the triggerer, workers are freed.

C Faster polling

More load, the same occupied slots.

D One big task

Not idempotent, and one failure blocks every file.

THE SIGNAL

Many long waits filling the workers: deferrable operators.

> A new DAG with start_date one year ago and catchup=True was deployed this morning.
>
> It immediately launched 365 runs, overloading the warehouse. Only the last 7 days actually need processing.

What should the team do?

A Set catchup=False, then backfill only the last seven days

B Move the start date to tomorrow, and skip the missing week

C Keep catchup on, but limit the DAG to one active run

D Delete the DAG, and run the seven days by hand in BigQuery

◆ PAUSE AND DECIDE

What should the team do?

A Catchup off, backfill seven

Exactly the runs needed, deliberately.

B Start tomorrow

The week that needs processing never runs.

C One active run

Still a year of runs, just in a slow line.

D Run by hand

Loses retries, history and the DAG itself.

THE SIGNAL

Too many historical runs: catchup off, then a deliberate backfill.

Module 35 in five lines

- **Airflow** for data pipelines, **Workflows** for chaining services, **Scheduler** for one timer.
- Tasks are **idempotent**: one step each, overwrite by partition, with retries.
- **Deferrable** operators wait in the triggerer, freeing workers.
- Catchup **off**, then backfill on purpose. Fixed start dates.
- Heavy work out of Airflow. **Gen 3** is current, Gen 1 is end of life.

> COMING UP
>
> Next: module 36, monitoring and troubleshooting.